

# Řetězce

Karel Richta a kol.

Katedra technických studií  
Vysoká škola polytechnická v Jihlava

© Richta, 2022

Základy strukturovaného programování, ZSP  
11/2022, Lekce 8

<https://moodle.vspj.cz/course/view.php?id=202262>



# Řetězce v C <string.h> (<cstring>)

```
size_t strlen(char *Retezec);
char *strcpy(char *Cil, char *Zdroj);
char *strncpy(char *Cil, char *Zdroj, size_t MaxDelka);
char *strcat(char *Cil, char *Zdroj);
char *strncat(char *Cil, char *Zdroj, size_t MaxDelka);
int strcmp(char *Prvni, char *Druhy);
int strncmp(char *Prvni, char *Druhy, size_t N);
char *strchr(char *Retezec, int Znak);
char *strrchr(char *Retezec, int Znak);
char *strstr(char *Retezec, char *Podretezec);
void *memcpy(void *Cil, void *Zdroj, size_t Delka);
void *memmove(void *Cil, void *Zdroj, size_t Delka);
int memcmp(void *Prvni, void *Druhy, size_t Delka);
void *memchr(void *Adresa, int Byte, size_t Delka);
void *memset(void *Adresa, int Byte, size_t Delka);
```

## Makro:

NULL - jako celočíselná konstanta (0 nebo 0L), nebo hodnota typu nullptr\_t  
(nullptr) - (void\*)0

Typ: size\_t - typ pro délku libovolného objektu (jako unsigned int)

# Vytvořme si sofistikované řetězce

```
struct String {  
    int pocet; // aktuální délka řetězce  
    char *znaky; // ukazatel na dyn. alokované pole  
};
```

nebo lépe:

```
typedef struct String {  
    int pocet; // aktuální délka řetězce  
    char *znaky; // ukazatel na dyn. alokované pole  
} Strings;
```

```
String *create(); // vytvoří prázdný řetězec  
String *copy(const String*); // vytvoří řetězec jako kopii jiného  
String *conv(const char *); // konverze z C-Stringu  
void drop(String*); // zruší řetězec (destruktor)  
int delka(String*); // vrátí délku řetězce  
void putString(FILE *, String*); // připojí řetězec do souboru
```

# Implementace operací I.

```
Strings *create(void) { // vytvoří prázdný řetězec
    Strings *res =
        (Strings *)malloc(sizeof(Strings));
    res->pocet = 0;
    res->znaky = (char *)calloc(sizeof(char), 1);
    memory += sizeof(Strings) + 1;
    return res;
}

int delka(const Strings* str) { // vrátí délku řetězce
    return str->pocet;
}
```

# Implementace operací II.

```
Strings *conv(const char *src) { // konverze z C-Stringu
    Strings *res = create();
    res->pocet = (int)strlen(src);
    res->znaky = (char *)calloc(sizeof(char), res->pocet + 1);
    strncpy(res->znaky, src, res->pocet);
    memory += sizeof(Strings) + res->pocet + 1;
    return res;
}

void drop(Strings* str) { // zruší řetězec (destruktor)
    free(str->znaky);
    memory -= (str->pocet + 1 + sizeof(Strings));
    free(str);
    return;
}
```

## Pomocné služby II.

```
int memory = 0; // zabrane místo na halde

void printString(FILE *f, const Strings* str) {
    // výstup řetězce do streamu
    fprintf(f, "%s", str->znaky);
    return;
}

void printSize(const Strings *s) {
    printf("Objekt: %p ma hodnotu: %s\n", s, s->znaky);
}

void check(const char *s) {
    printf(" %s obsazeno na halde: %d bytu\n", s, memory);
}
```

# Příklad použití

```
#include "Strings.h"

int main() {
    check("Na zacatku");
    Strings* s1 = conv("Prace s retezci\n");
    printString(stdout, s1);
    check("Po vytvoreni s1"); printSize(s1);
    Strings* s2 = create();
    printString(stdout, s2);
    check("Po vytvoreni s2"); printSize(s2);
    s2 = conv("\n");
    printString(stdout, s2); printSize(s2);
    check("Po doplneni s2");
    s2 = copy(s1);
    printString(stdout, s2); printSize(s1); printSize(s2);
    check("Po zkopirovani s1 do s2");
    drop(s1); check("Po zruseni s2");
    drop(s2); check("Po zruseni s1");
}
```

# Výsledek

```
Na zacatku obsazeno na halde: 0 bytu
Prace s retezci
Po vytvoreni s1 obsazeno na halde: 50 bytu
Objekt: 000001E4901FF6E0 ma hodnotu: Prace s retezci
Objekt: 000001E4901FF140 ma hodnotu:
Po vytvoreni s2 obsazeno na halde: 67 bytu
Objekt: 000001E4901FF190 ma hodnotu:
Po doplneni s2 obsazeno na halde: 102 bytu
Prace s retezci
Objekt: 000001E4901FF6E0 ma hodnotu: Prace s retezci
Objekt: 000001E4901FF280 ma hodnotu: Prace s retezci
Po zkopirovani s1 do s2 obsazeno na halde: 152 bytu
Po zruseni s2 obsazeno na halde: 119 bytu
Po zruseni s1 obsazeno na halde: 86 bytu
```



# The End